

Introduction to Parallel Computing

PARCO — Lecture 01

Introduction to Parallel Computing

Supercomputers

The current supercomputers (Top500) referenced in the lecture:

System	Site / Country
JUPITER FZJ	Germany
HPC6 ENI	Italy
LUMI CSC	Finland
Alps CSCS	Switzerland
Leonardo CINECA	Italy
El Captain	Livermore National Lab, US
Frontier	Oak Ridge National Lab, US
Aurora	Argonne National Lab, US
Eagle	Microsoft
Fugaku Riken	Japan

The FLOPs scale

FLOPS floating-point operations per second.

The scale, each step a 1000x jump: 10^{12} teraFLOPS $\rightarrow 10^{15}$ petaFLOPS $\rightarrow 10^{18}$ exaFLOPS $\rightarrow 10^{21}$ zettaFLOPS?

- Exascale machine $\rightarrow$ the 10^{18} tier.
- It is estimated that the brain exceeds 10^{16} synaptic operations per second.
- Each 1000x jump changes which problems become tractable.

What problems they solve

Physical simulations	simulations	Climate and environment	envi- Drug discovery
Artificial intelligence (AI)	intelligence (AI)	Data analytics and big data	Forecasting and prediction

Anatomy: from nodes to a machine

A node = a powerful server. Every node is built from processors (CPU), accelerators (GPUs), a memory hierarchy and interconnects (Network Interface Card).

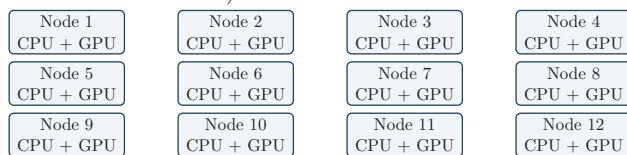


Fig. 1: Each node solves a sub-problem; multiple nodes work together to solve the entire large problem.

Key idea Parallelism: multiple nodes work together to solve the entire large problem.

Inside the node

CPU (control + logic)	Memory	GPU / other accelerators
Few, highly versatile cores; excellent for running many different applications via the operating system; the CPU coordinates the workload	RAM + HBM; DMA	Massive parallel acceleration; thousands of simple cores; excellent for structured data like images; accelerates AI and simulations

Network NICs interconnect the node. Supercomputers typically have both, enabling shared-memory programming models.

From the node to the rack

A rack integrates multiple nodes, power and sockets, a cooling system, switches and network components.

Why it is needed It makes the system manageable: space, energy, cabling, and maintenance.

From racks to topology

Goal	Low latency, high-bandwidth, multi-path to avoid congestion and bottlenecks.
Network topology	Different shapes for different needs and applications.
Examples	Mesh, torus, fat-tree, dragonfly.

Fully stacked system

The supercomputer looks like and acts as a single machine.

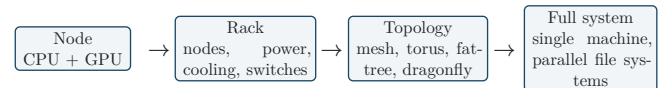


Fig. 2: Nodes, racks, and network storage work together.

- Infrastructure: energy and cooling, resources management.
- Parallel file systems.
- Message Passing Programming Models to design distributed-memory applications.

Questions the course wants to address

- What do we need to know about the system to design efficient algorithms?
- How to program a single parallel system?
- What are the metrics and goals we are interested in?
- How to program multiple parallel machines?
- How can we coordinate all these activities on such a complex system?

Topics covered in the course

Parallel architectures	Multicore, accelerators, interconnects.
Parallel programming models	Shared memory, distributed memory, MPI.
Parallel algorithms	Decomposition, patterns, scientific computing, AI/data science.
Performance	Speedup, efficiency, scalability, communication, portability/productivity.

What this course is about

We will not design new parallel architecture but adopt a Hardware-Software co-design approach.

- Traditionally parallelism has been addressed from an architecture perspective.
- Today performance depends on several components.

In this course (80%)	Algorithm design, data decomposition; Libraries and Primitives; Programming models and implementations.
20%	Run-time and Compilers; Architecture and Network Interconnection.

Parallel Architectures and Parallel Computing

Serial computing

- A program is broken into a sequence of logic instructions.
- Instructions are executed sequentially one-by-one.
- Executed on a single processor.
- Only one instruction may execute at any moment in time.

Motivation: e-commerce logistics

Suppose an e-commerce company and its logistics. Boxes of equal size that contain goods and products are organized into a big storage. An employee P should take the box from a storage area and move them to a load area to be shipped through a truck service.

Task decomposition as sequential steps:

- P takes a box from the storage (2 sec per kg);
- P goes to the area for the loading (10 m per second);
- P drops the box (2 sec);
- P comes back to the storage (10 m per second).

Class discussion Compute the total time assuming that the distance from the storage to the load area is 100 m?

With a 100 m distance: $2 + 10 + 2 + 10 = 24$ seconds for each 1 kg box. If your truck contains 100 boxes, P needs 2400 seconds!

Fact Parallel computing allows us to reduce time-to-solution, increase throughput, or solve larger problems by exploiting multiple computing resources.

Possible solutions

Hire a faster and stronger employee	Increase the clock of the processors to reduce the latency and increase the throughput.
Buy and use a cart or a forklift	Vectorization, wider data paths, batching, or higher memory bandwidth.
Hire more employees	Multiple processing elements to increase the throughput.

Other solutions: change the entire system organization.

1. Adding more independent helpers (functional units).
2. Reduce the path from the storage to the loading area (reducing the path length).
3. Add smaller temporary storage in the path (cache).

Transistor technology and Dennard scaling

Performance of individual functional units depends on transistor technology.

Transistor linear dimensions could be scaled down by 30% (0.7x):

- their area decreases by approximately 50% (0.5x);
- the voltage, V , decreases by 30% (0.7x) to keep the electric field approximately constant;
- capacitance, C , decreases by approximately 30% (0.7x).

Other implications:

- circuit delay decreases by approximately 30% (0.7x), allowing operating frequency, f , to increase by about 40% (1.4x);
- dynamic power per transistor decreases by approximately 50% (0.5x);
- with approximately 2x transistor density, power density remains approximately constant (ideal Dennard scaling).

Dynamic power $P \propto C \cdot V^2 \cdot f$. Under ideal Dennard scaling: $0.7 \times 0.7^2 \times 1.4 \approx 0.5$ per transistor.

Moore's law and performance per watt

Moore's law says that the number of transistors doubles approximately every two years; in combination with the Dennard observations.

Empirical observation Performance per watt grows even faster, doubling about every 18 months (1.5 years).

Why not keep increasing clock frequency?

Transistors are getting smaller and faster:

- more efficient processors (energy consideration);
- we can put more cores in the same die.

Main design constraint: cost and power

Single-core CPUs	Complex control hardware. Pro: flexibility + performance. Cons: expensive (cost and power).
Multi/Many-core	Simpler control hardware. Pros: replicating processing resources can provide better throughput and performance/W than spending the transistor budget only on increasing single-core complexity. Cons: more restrictive/complex programming models and algorithm design.

From More Cores to Parallel Execution

Hardware and software perspectives

Hardware	Power and cost constraints limit further single-core scaling; multi/many-core architectures provide multiple processing elements (PEs); more PEs increase the potential amount of work that can be executed simultaneously.
Software	Having multiple cores is not sufficient to obtain speedup; the application must expose multiple activities that we can process independently; these units of work expose concurrency.

Key idea Parallel hardware provides the resources; concurrency in the algorithm provides the work to exploit them.

What is parallel computing

Parallel Computing Parallel Computing is the simultaneous use of multiple compute resources to solve a computational problem.

- A program is broken into discrete parts that can be solved concurrently.
- Multiple instructions are executed at any moment in time, depending on the number of processors.
- An overall control/coordination mechanism is employed.
- Each PE executes its own instruction stream while multiple PEs execute simultaneously.

The program should be able to:

- decompose the pieces of work that can be solved simultaneously;
- execute multiple instructions at any moment in time.

The goal is to solve the problem in less time by exploiting multiple compute resources.

From sequential to parallel algorithms

Algorithms are a recipe for solving a problem through a sequence of simple operations. The larger and more complex the problem, the more time it takes to solve. A problem is split into Part 1, Part 2, Part 3, Part 4 and then produces the result.

Example: preparing pasta carbonara.

Algorithm / Recipe	1. Cook the pasta; 2. Scramble the eggs; 3. Pasteurize the eggs; 4. Guanciale; 5. Assemble.
---------------------------	---

Independent actions Cook the pasta and scramble the eggs.

Dependent actions Scramble the eggs and pasteurize them.

Concurrency exposes the independent parts (Part 1 next to Part 2, Part 3, Part 4), producing time saved. Scaling to 200 plates: the same dependency structure is replicated per dish and distributed across several chefs.

Task vs Data parallelism

Task parallelism Different jobs at the same time.

Data parallelism The same operation on many data items.

Scalability How much performance improves as the number of nodes increases.

Question Why could we not scale?

Data and task decomposition

We can consider parallelism on the data or based on the type of activity:

- Data decomposition: the data is decomposed in small chunks. "Each processing element (PE) performs a specific task on a different portion of the data".
- Task decomposition: the problem is decomposed according to the work that must be done. "Each processing element (PE) performs a different task on a portion or on all the data".

Concurrency and parallelism

Concurrency (degree of) Multiple activities whose execution can overlap in time.

Parallelism (degree of) Multiple activities actually executing simultaneously.

Challenges Not all tasks are embarrassingly parallel; we do not have infinite resources.

Amdahl's law

95% parallel, 5% sequential.

- Even a few serial steps can slow down everything.

- Communication and synchronization have a cost.
- Good algorithm > simply adding hardware.

Execution-time model

- 1 processing element: serial sT_1 ; parallelizable $(1-s)T_1$.
- p processing elements: serial sT_1 ; parallelizable $(1-s)\frac{T_1}{p}$.

$$T_p = sT_1 + (1-s)\frac{T_1}{p}$$

$$S(p) = \frac{T_1}{T_p} = \frac{1}{s + \frac{1-s}{p}}$$

Upper bound: $\lim_{p \rightarrow \infty} S(p) = \frac{1}{s}$

s = serial fraction of the one-PE execution time; $1-s$ = perfectly parallelizable fraction; p = number of processing elements. Ideal model: fixed problem size, perfect scaling of the parallel fraction, and no additional parallel overhead.

Key idea The serial fraction sets a hard speedup limit.

From code to Amdahl: worked example

```
read_input(A, N)           // 10 s - sequential
for i = 0 ... N-1          // 85 s - parallelizable
    B[i] = expensive_function(A[i])
write_result(B, N)         // 5 s - sequential
```

Measured on one PE: $T_1 = 10 + 85 + 5 = 100$ s. Sequential time $T_s = 10 + 5 = 15$ s, so $s = \frac{15}{100} = 0.15$. Parallelizable time $T_{\text{par}} = 85$ s, so $1-s = 0.85$. Speedup model: $S(p) = \frac{1}{0.15 + \frac{0.85}{p}}$ (serial fraction = 15%).

Question to ask first Which regions can actually execute concurrently?

Given $T_1 = 100$ s, $s = 0.15$, $1-s = 0.85$, $T_p = 15 + \frac{85}{p}$:

	1 PE	4 PEs	16 PEs	\$\infty\$ PEs
Execution time	100.0 s	36.25 s	20.31 s	15.0 s
Speedup	1.00x	2.76x	4.92x	6.67x

Execution time cannot fall below the serial work. Adding processors gives diminishing returns: once the parallel part is very fast, the unchanged serial part dominates the total time.

Parallel Computing and Applications

From observations to parallel code

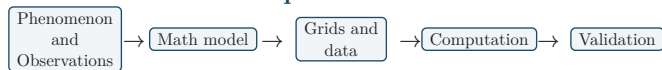


Fig. 3: From phenomenon and observations to validated computation.

Climate

- Atmosphere and oceans divided into millions of cells.
- Each cell evolves and exchanges information.
- Higher resolution = more detailed forecasts.

Example: hurricanes, heatwaves, droughts, air quality.

Computational Fluid Dynamics

$$\text{Equation } \rho \left(\frac{\partial u}{\partial t} + (u \cdot \nabla) u \right) = -\nabla p + \mu \nabla^2 u + f$$

Meaning of symbols: $u \rightarrow$ fluid velocity; $p \rightarrow$ pressure; $\mu \rightarrow$ viscosity; $\rho \rightarrow$ density. The equations describe fluid motion.

Computational Fluid Dynamics applications:

- Simulation of the airflow around the car.
- Reduction of aerodynamic drag.
- Optimization of downforce.
- Turbulence analysis.
- Improvement of stability and efficiency.

Abstraction and Parallelism

Modern parallel architectures

- Multi processors/multi-core architectures with bus interconnections (e.g., Intel, AMD, ARM, and RISC-V based processors).
- Multi processors/multi-core systems and interconnects.
- Shared-memory vs Distributed Memory.
- Accelerators and domain-specific architectures (e.g., Graphics Processing Units, Tensor Processing Unit, Dataflow architectures).

All of them are parallel architectures. What are the main differences you see? How to program them?

The abstract parallel machine

We can consider an abstract parallel machine:

- Programming models allow to expose features at a higher level of abstraction (e.g., computational unit, memory).
- Ignore the implementation of a specific set of features (these features can be implemented in hardware or with efficient algorithms).

From this perspective we need to consider the dualism software / hardware. Examples:

- Data type vs physical representation of the words in the memory: in C, you declare an integer without knowing the specific representation (for example little or big endian).
- Threads and Processes vs Physical Cores.
- Barriers and synchronization mechanism vs Memory Transactions and Atomic operations.

Performance Considerations

Parallel computing and performance

Performance is the measure of the completion of a task according to a specific measure.

- Metrics are, e.g., time, power consumption, resource utilizations etc.
- Theoretical computational models.
- Composed metrics: operation per time unit (instructions, floating point operations, memory access), efficiency, latency, scalability (weak and strong).
- Domain specific units (e.g., inference per time unit, edges per time unit).

Portability, performance portability, productivity

Portability	The capacity of software to run on different architectures or environments. OpenCL provides a programming model and level of abstraction to match different architectures.
Performance portability	The ability of the software to achieve a “consistent” level of performance across multiple architectures.
Productivity	The effort required to develop, debug, tune, and maintain parallel software.

Productivity via parallel programming models (add more abstractions into a standard); productivity via domain-specific programming models (e.g., identification algorithm primitives).

Takeaway

1. Technology limits have made parallelism essential.
2. Parallel architectures are entering a new golden age.
3. Algorithms must be redesigned to expose parallelism.
4. We need to understand how to decompose them and coordinating task.
5. HPC enables problems to be solved at unprecedented scale.
6. Scalability depends on communication, synchronization, and data locality.